crafting-interpreters

#personal #book #book-summary

General Information

Book Information:

- Title: Crafting Interpreters
- Author: Robert Nystrom

Book Summary

1 - Introduction

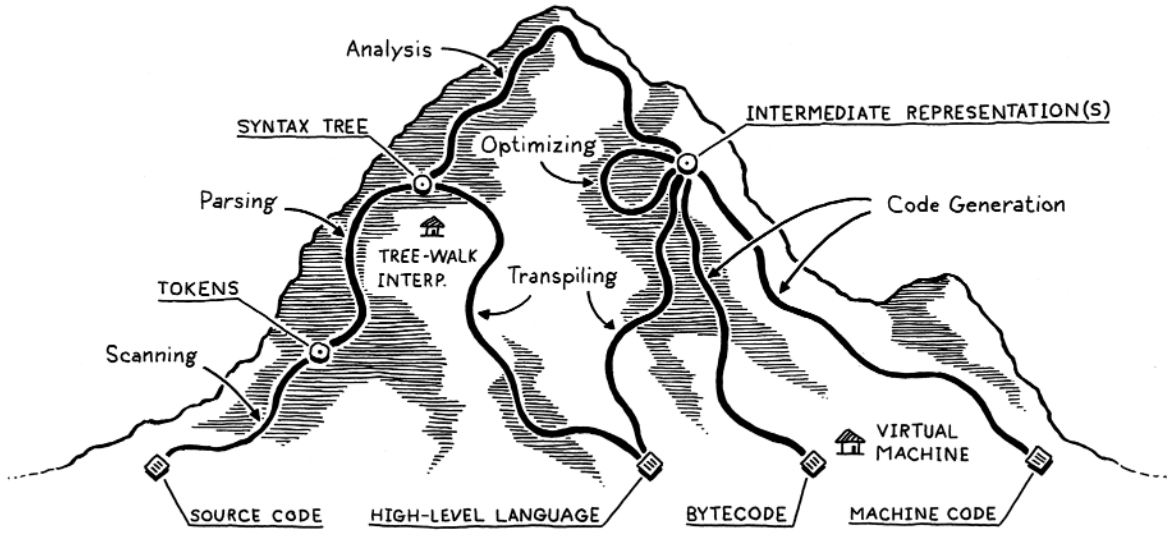
- We will build interpreter for a language called "Lox"
- Interpreters, although complex, are not all that hard to implement
- Book is broken into three parts
    1. Introduction / Get up to speed
    2. Build Lox Interpreter (Java)
    3. Build on existing Lox interpreter in C (C)
- This book does not use tools like Lex and Yacc. Everything is done from scratch to get better understanding.
- This book writes it's code in Java
    - High level enough, but still pretty explicit in nature
    - Object-oriented langauage
    - Popular language

2 - A Map of the Territory

Parts of a Language

- Most languages are similar, although they might have subtle differences
- While language designers have found a lot, they tend to all use the same set of tools to implement the language
- The paths that an implementation may choose is a bit like climbing a mountain
  - Start off at the bottom with user's source code
  - When we reach the peek, we have a birds-eye view of what the user's code is asking for

- As we go down the other side of the mountain, we transform the user's high-level input into machine code in some manner



- Parts of a language:
  1. Lexing/Scanning: Takes in a linear stream of characters and chunks them together into something similar to words
     - These words are called "Tokens"
  2. Parsing: Takes the flat sequence of tokens and builds a tree structure that mirrors the nested nature of the grammar
  - This is called a **parse tree** or **abstract syntax tree** (AST)
  3. Analysis:
     - Static Analysis:
       - Binding/Resolution: where we find out where identifiers are defined and their values. This is where scope comes into play.
       - Scope: The region of source code where a certain name can be used to refer to a declaration
       - Type Check: We check types of variables and ensure that operations are using valid types.
         - If we have any invalid types, then we throw a **type error**.
       - Saving semantic information
         - stored as attributes on the syntax tree itself
         - store data in lookup table off to the side
         - Transform tree into another data structure (next section)
  4. Intermediate Representations: way that code is stored between source and destination forms
     - Interface, of sorts, between the two forms
     - Helps support multiple source languages and target platforms

5. Optimization: Swapping innefficent elements of a user's program out with more efficent segments
6. Code Generation: Converting the code from the intermediate representation to something that the computer or CPU can run
    - p-code or bytecode: code created for a virtual machine / not physical architecture
7. Runtime: Running the code on the machine. Services the language provides the program as it is running
    - In fully compiled language, the code implementing the runtime gets inserted into the resulting executable
        - In GO, every program has it's own copy of GO's runtime built-in

Shortcuts and Alternate Routes

1. **single-pass compilers**: interleaving parsing, analysis, and code generation in one step
    - restrict the design of the language. You lose the intermediate data structures that store program information
2. **Tree-walk Interpreter**: Program begins executing after forming AST. Traverses the syntax tree.
3. **Transpilers**: Taking source code in one language (your language) and changing it to another language.
    - For example, we could transpile from dillon-lang to javascript or python
4. **Just-in-time Compilation (JIT)**: Compiling on the fly based on the architecture of the user's computer hardware

Compilers and Interpreters

3 - The Lox Language

- Syntax derived from C (in the C family)
    - Looks a lot like javasript
- Language is dynamically-typed: vars can hold values of any type
- There are two main ways to manage memory:
    - Reference Counting
    - Garbage Collection
- Data Types:
    - Boolean
    - Numbers
    - Strings

- NIL (null)

4 - Scanning

The first step in any compiler **or** interpreter is scanning.

- Take in raw source code and groups it into meaningful chunks called tokens
- We then feed these to the parser

6 - Parsing Exmpressions

Context Free Grammar: A set of recursive rules used to generate patterns of strings. Context free grammars can describe all regular languages, and more, but not all languages.

- Grammars are descriptions of languages that provide a means for generating all possible strings contained in a language.

Example - All three letter binary numbers:
binary_number -> binary_digit binary_digit binary_digit
binary_digit -> 0 | 1